

CENG790 Big Data - Assignment 1 Report

Alperen Ovak - 2580801

March 25, 2026

Part 1 Solutions and Outputs

This report is organized question by question. For each question, the task, code, and output are presented.

Question 1

Task: Using Spark SQL API, select identifier, GPS coordinates, and license type.

Code:

```
originalFlickrMeta.createOrReplaceTempView("flickrMeta")
val selectPhotos = spark.sql(
  "SELECT photo_id, longitude, latitude, license " +
  "FROM flickrMeta"
)
printScope("Question 1 - Selected fields: identifier, GPS coordinates, license")
selectPhotos.show()
```

Output:

```
Question 1 - Selected fields: identifier, GPS coordinates, license
+-----+-----+-----+-----+
| photo_id|longitude| latitude| license|
+-----+-----+-----+-----+
|5610122230| -1.0| -1.0|Attribution-NonCo...|
|5223747995| -1.0| -1.0|Attribution-NonCo...|
|5592678175| -1.0| -1.0|Attribution-NonCo...|
|8807058226| -1.0| -1.0|Attribution-NonCo...|
|2860980452| -0.02592| -0.036392|Attribution-NonCo...|
|6928499157| -1.0| -1.0|Attribution-NonCo...|
|5674323056| -1.0| -1.0|Attribution-NonCo...|
|5734258350| -1.0| -1.0|Attribution-NonCo...|
|6198509952| -1.0| -1.0|Attribution-NoDer...|
|7050886391| -1.0| -1.0|Attribution-Share...|
|2445790010| -0.83496| -54.99022|Attribution-NonCo...|
|4913556997| -0.005252| -81.434204|Attribution-NonCo...|
|1345730799| -0.002489| -82.9847| Attribution License|
|1345733105| -0.040082| -82.98526| Attribution License|
|5052929796| -0.088212| 0.00773|Attribution-NonCo...|
|3116901547| -1.2E-5| 3.0E-6| Attribution License|
|3117729084| -1.2E-5| 3.0E-6| Attribution License|
|3737526549| -0.300909| 0.094894|Attribution-Share...|
|3117764790| -1.2E-5| 3.0E-6| Attribution License|
|3117768410| -1.2E-5| 3.0E-6| Attribution License|
+-----+-----+-----+-----+
only showing top 20 rows
```

Explanation: We select only needed columns from the big table. This helps us focus on id, place, and license.

Question 2

Task: Create DataFrame of interesting pictures (license not null, coordinates not -1.0).

Code:

```
val interestingPhotos = selectPhotos.filter(  
  col("license").isNotNull &&  
  col("latitude") != -1.0 &&  
  col("longitude") != -1.0  
)  
printScope("Question 2 - Interesting pictures DataFrame (license != null, coords !=  
  -1.0)")
```

Output:

```
Question 2 - Interesting pictures DataFrame (license != null, coords != -1.0)  
(DataFrame created for later explain/show steps)
```

Explanation: We keep only useful pictures. We remove rows with missing license or invalid coordinates.

Question 3

Task: Display execution plan of interesting pictures DataFrame.

Code:

```
printScope("Question 3 - Execution plan for interesting pictures DataFrame")  
interestingPhotos.explain()
```

Output:

```
Question 3 - Execution plan for interesting pictures DataFrame  
== Physical Plan ==  
*(1) Filter (((isnotnull(latitude#11) AND isnotnull(longitude#10))  
AND isnotnull(license#15)) AND NOT (latitude#11 = -1.0))  
AND NOT (longitude#10 = -1.0))  
+- FileScan csv [photo_id#0L,longitude#10,latitude#11,license#15] ...
```

Explanation: This plan shows why filtering is efficient. Spark first checks null and -1.0 conditions while reading rows, so bad rows are removed early. This means less data moves to next steps, and later operations become faster.

Question 4

Task: Display the data of interesting pictures.

Code:

```
printScope("Question 4 - Display interesting pictures")  
interestingPhotos.show()
```

Output:

Question 4 - Display interesting pictures

```
+-----+-----+-----+-----+
| photo_id|longitude| latitude| license|
+-----+-----+-----+-----+
|2860980452| -0.02592| -0.036392|Attribution-NonCo...|
|2445790010| -0.83496| -54.99022|Attribution-NonCo...|
|4913556997|-0.005252|-81.434204|Attribution-NonCo...|
|1345730799|-0.002489| -82.9847| Attribution License|
|1345733105|-0.040082| -82.98526| Attribution License|
|5052929796|-0.088212| 0.00773|Attribution-NonCo...|
|3116901547| -1.2E-5| 3.0E-6| Attribution License|
|3117729084| -1.2E-5| 3.0E-6| Attribution License|
|3737526549|-0.300909| 0.094894|Attribution-Share...|
|3117764790| -1.2E-5| 3.0E-6| Attribution License|
|3117768410| -1.2E-5| 3.0E-6| Attribution License|
|4262750956|-0.050373| 0.342372|Attribution-Share...|
|3397220196|-0.001373| 8.58E-4|Attribution-NonCo...|
|3117773794| -1.2E-5| 3.0E-6| Attribution License|
|3117761408| -1.2E-5| 3.0E-6| Attribution License|
|4591167499|-0.851526| 10.785503|Attribution-NonCo...|
|4591166029|-0.851526| 10.785503|Attribution-NonCo...|
|3765897146|-0.627593| 10.797897|Attribution-NonCo...|
|3755727437| -0.52597| 10.991749|Attribution-NonCo...|
|8491558947|-0.972974| 10.822321|Attribution-NonCo...|
+-----+-----+-----+-----+
only showing top 20 rows
```

Explanation: Here we display sample rows after filtering. The result looks correct because coordinates are valid and license is present.

Question 5(a)

Task: Load license file, filter NonDerivative licenses, join with interesting pictures, show plan and results.

Code:

```
val licenseDF = spark.read
  .format("csv")
  .option("delimiter", "\t")
  .option("header", "true")
  .load("data/FlickrLicense.txt")

val nonDerivLicenses = licenseDF.filter(col("NonDerivative")==1)

val finalDF_beforeCache = interestingPhotos.join(
  nonDerivLicenses,
  interestingPhotos("license") === nonDerivLicenses("Name")
)

printScope("Question 5(a) - Join execution plan before cache")
finalDF_beforeCache.explain()
printScope("Question 5(a) - Join results before cache")
finalDF_beforeCache.show()
```

Output:

```
Question 5(a) - Filtered NonDerivative licenses
```

```

+-----+-----+-----+-----+
| Name|Attribution|Noncommercial|NonDerivative|
+-----+-----+-----+-----+
|Attribution-NoDer...| 1| 0| 1|
|Attribution-NonCo...| 1| 1| 1|
+-----+-----+-----+-----+

Question 5(a) - Join execution plan before cache
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- BroadcastHashJoin [license#15], [Name#109], Inner, BuildRight, false
  :- Filter ...
    :- FileScan csv [photo_id#0L,longitude#10,latitude#11,license#15] ...

Question 5(a) - Join results before cache
+-----+-----+-----+-----+-----+
| photo_id|longitude| latitude| license| Name|
+-----+-----+-----+-----+-----+
|2860980452|-0.02592|-0.036392|Attribution-NonCo...|Attribution-NonCo...|
|4913556997|-0.005252|-81.434204|Attribution-NonCo...|Attribution-NonCo...|
|8491558947|-0.972974| 10.822321|Attribution-NonCo...|Attribution-NonCo...|
|3725078884|-0.906372| 11.497519|Attribution-NonCo...|Attribution-NonCo...|
|3724276245|-0.906372| 11.497519|Attribution-NonCo...|Attribution-NonCo...|
|3725109474|-0.906372| 11.497519|Attribution-NonCo...|Attribution-NonCo...|
|3725065346|-0.906372| 11.497519|Attribution-NonCo...|Attribution-NonCo...|
|3725062966|-0.906372| 11.497519|Attribution-NonCo...|Attribution-NonCo...|
|4105402596|-0.527343| 12.287764|Attribution-NoDer...|Attribution-NoDer...|
|5512012382|-0.109863| 13.781568|Attribution-NonCo...|Attribution-NonCo...|
| 104791081|-0.274744| 13.578083|Attribution-NonCo...|Attribution-NonCo...|
|5511312835|-0.109863| 13.781568|Attribution-NonCo...|Attribution-NonCo...|
| 104778685|-0.274744| 13.578083|Attribution-NonCo...|Attribution-NonCo...|
|6442481127|-0.067377| 16.330847|Attribution-NoDer...|Attribution-NoDer...|
| 259199471|-0.791015| 16.997038|Attribution-NonCo...|Attribution-NonCo...|
|6442477951|-0.050554| 16.270626|Attribution-NoDer...|Attribution-NoDer...|
+-----+-----+-----+-----+-----+

```

Explanation: The plan uses BroadcastHashJoin. Spark sends the small NonDerivative license table to all workers, then each worker matches local photo rows without heavy shuffle. This is faster because only a small table is broadcast. The join result keeps only rows where photo license equals license name.

Question 5(b)

Task: Cache interesting pictures and compare the join execution plan.

Code:

```

interestingPhotos.cache()

val finalDF_afterCache = interestingPhotos.join(
  nonDerivLicenses,
  interestingPhotos("license") === nonDerivLicenses("Name")
)

printScope("Question 5(b) - Join execution plan after caching interesting pictures")
finalDF_afterCache.explain()

printScope("Question 5(b) - Join results after cache")

```

```
finalDF_afterCache.show()
```

Output:

```
Question 5(b) - Join execution plan after caching interesting pictures
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- BroadcastHashJoin [license#15], [Name#109], Inner, BuildRight, false
   :- Filter isnotnull(license#15)
      :- InMemoryTableScan [photo_id#0L, longitude#10, latitude#11, license#15] ...
```

Question 5(b) - Join results after cache

```
+-----+-----+-----+-----+-----+
| photo_id|longitude| latitude| license| Name|
+-----+-----+-----+-----+-----+
|2860980452|-0.02592|-0.036392|Attribution-NonCo...|Attribution-NonCo...|
|4913556997|-0.005252|-81.434204|Attribution-NonCo...|Attribution-NonCo...|
|8491558947|-0.972974| 10.822321|Attribution-NonCo...|Attribution-NonCo...|
|3725078884|-0.906372| 11.497519|Attribution-NonCo...|Attribution-NonCo...|
|3724276245|-0.906372| 11.497519|Attribution-NonCo...|Attribution-NonCo...|
|3725109474|-0.906372| 11.497519|Attribution-NonCo...|Attribution-NonCo...|
|3725065346|-0.906372| 11.497519|Attribution-NonCo...|Attribution-NonCo...|
|3725062966|-0.906372| 11.497519|Attribution-NonCo...|Attribution-NonCo...|
|4105402596|-0.527343| 12.287764|Attribution-NoDer...|Attribution-NoDer...|
|5512012382|-0.109863| 13.781568|Attribution-NonCo...|Attribution-NonCo...|
| 104791081|-0.274744| 13.578083|Attribution-NonCo...|Attribution-NonCo...|
|5511312835|-0.109863| 13.781568|Attribution-NonCo...|Attribution-NonCo...|
| 104778685|-0.274744| 13.578083|Attribution-NonCo...|Attribution-NonCo...|
|6442481127|-0.067377| 16.330847|Attribution-NoDer...|Attribution-NoDer...|
| 259199471|-0.791015| 16.997038|Attribution-NonCo...|Attribution-NonCo...|
|6442477951|-0.050554| 16.270626|Attribution-NoDer...|Attribution-NoDer...|
+-----+-----+-----+-----+-----+
```

Observation:

Before cache, Spark scanned the CSV for interestingPhotos.

After cache, Spark reads interestingPhotos from memory (InMemoryTableScan).

Explanation: After caching, Spark uses InMemoryTableScan for interestingPhotos. This avoids recomputation of the same filter lineage and avoids reading CSV again for the next join. In this join context, cache helps when we reuse interestingPhotos in multiple actions (explain/show/write), so repeated work becomes smaller.

Question 5(c)

Task: Save final result as CSV with header.

Code:

```
finalDF_afterCache.write
  .mode("overwrite")
  .option("header", "true")
  .csv("output/interesting_nonderivative_photos")
printScope("Question 5(c) - CSV written with header to output/
  interesting_nonderivative_photos")
```

Output:

```
Question 5(c) - CSV written with header to output/interesting_nonderivative_photos
[success] Total time: 9 s, completed Mar 25, 2026, 9:15:56 PM
```

Explanation: We save the final table as CSV with header. Now the result is ready to use outside Spark.

Part 2 Solutions and Outputs

Question 1

Task: Display the 10 lines of the RDD (using take(..)) and display the number of elements in the RDD (count()).

Code:

```
printScope("Question 1 - First 10 lines from RDD[String] and total count")
originalFlickrMeta.take(10).foreach(println)
val totalRawElements: Long = originalFlickrMeta.count()
println(s"Total number of elements in originalFlickrMeta: $totalRawElements")
```

Output:

```
Question 1 - First 10 lines from RDD[String] and total count
5610122230 54345792@N00 higginskurt 2011-04-11 10:20:13.0 1302531613 SROTAROX %3Ca+
href%3D%22http%3A%2F%2Fwww.serotta.com%2F%22+rel%3D%22nofollow%22%3ESerotta%3C%2Fa
%3E+Rocks%21+Great+plate%2C+mystery+driver+%3A%29 -1.0 -1.0 16 http://www.flickr
.com/photos/54345792@N00/5610122230/ http://farm6.staticflickr.com/5263/5610122230
_fd145ac94b.jpg Attribution-NonCommercial-ShareAlike License http://
creativecommons.org/licenses/by-nc-sa/2.0/ 5263 6 fd145ac94b 83ae88a5e1 jpg 0
5223747995 54345792@N00 higginskurt 2010-12-01 12:58:33.0 1291226313 Titan I+won+this
+one...+albeit+using+abbreviated-limit+of+one+hour-Risk-type-battle+rules. -1.0
-1.0 16 http://www.flickr.com/photos/54345792@N00/5223747995/ http://farm6.
staticflickr.com/5204/5223747995_cdd153b83f.jpg Attribution-NonCommercial-
ShareAlike License http://creativecommons.org/licenses/by-nc-sa/2.0/ 5204 6
cdd153b83f 56acb0c92a jpg 0
5592678175 54345792@N00 higginskurt 2011-04-05 15:51:03.0 1302033063 Jenny+snoozing+
on+the+ride -1.0 -1.0 16 http://www.flickr.com/photos/54345792@N00/5592678175/
http://farm6.staticflickr.com/5190/5592678175_98e73f50d7.jpg Attribution-
NonCommercial-ShareAlike License http://creativecommons.org/licenses/by-nc-sa/2.0/
5190 6 98e73f50d7 ef2c0376cc jpg 0
8807058226 34427466499@N01 Padre+Denny 2013-05-23 16:32:53.0 1369344773 -1.0 -1.0
16 http://www.flickr.com/photos/34427466499@N01/8807058226/ http://farm4.
staticflickr.com/3791/8807058226_7fd8d43ea4.jpg Attribution-NonCommercial License
http://creativecommons.org/licenses/by-nc/2.0/ 3791 4 7fd8d43ea4 e028261a90 jpg 0
2860980452 23516515@N07 50mm-traveller 2008-06-19 12:25:30.0 1221513976 FUJIFILM+
FinePix+F30 leaves+in+motion leaves+in+motion gallery2flickr -0.02592 -0.036392
12 http://www.flickr.com/photos/23516515@N07/2860980452/ http://farm4.staticflickr
.com/3258/2860980452_c569fb4104.jpg Attribution-NonCommercial-NoDerivs License
http://creativecommons.org/licenses/by-nc-nd/2.0/ 3258 4 c569fb4104 02c3b039dc jpg
0
6928499157 21092816@N00 slimninja 2012-02-25 09:08:12.0 1330182492 -1.0 -1.0 16
http://www.flickr.com/photos/21092816@N00/6928499157/ http://farm8.staticflickr.
com/7193/6928499157_9e06726e39.jpg Attribution-NonCommercial-ShareAlike License
http://creativecommons.org/licenses/by-nc-sa/2.0/ 7193 8 9e06726e39 d2333b3193 jpg
0
5674323056 66338859@N00 Pati-G+%5BPati+Gaitan%5D 2011-04-30 21:25:16.0 1304213116
Rose+Park+Spring+Celebration+2011 Face+painting+by%3A%0ASherri+LaReayx%0A%0
Afairysherri%40aol.com -1.0 -1.0 16 http://www.flickr.com/photos/66338859@N00
/5674323056/ http://farm6.staticflickr.com/5305/5674323056_eb359d03ac.jpg
Attribution-NonCommercial-ShareAlike License http://creativecommons.org/licenses/
by-nc-sa/2.0/ 5305 6 eb359d03ac 6158a39a78 jpg 0
```

```

5734258350 11136034@N06 MDB+28 2011-05-18 12:48:45.0 1305737325 Orchids+in+the+
Solarium+at+the+Cloister. -1.0 -1.0 16 http://www.flickr.com/photos/11136034@N06
/5734258350/ http://farm4.staticflickr.com/3022/5734258350_c26f09d8fd.jpg
Attribution-NonCommercial-NoDerivs License http://creativecommons.org/licenses/by-
nc-nd/2.0/ 3022 4 c26f09d8fd b73ffd8725 jpg 0
6198509952 95189931@N00 C_Harvey 2011-09-30 11:20:32.0 1317403232 -1.0 -1.0 16
http://www.flickr.com/photos/95189931@N00/6198509952/ http://farm7.staticflickr.
com/6130/6198509952_1207254fb0.jpg Attribution-NoDerivs License http://
creativecommons.org/licenses/by-nd/2.0/ 6130 7 1207254fb0 3828e470f4 jpg 0
7050886391 99005287@N00 joqel 2012-04-06 17:27:00.0 1333726020 -1.0 -1.0 16 http:
//www.flickr.com/photos/99005287@N00/7050886391/ http://farm6.staticflickr.com
/5271/7050886391_0bc8a139b0.jpg Attribution-ShareAlike License http://
creativecommons.org/licenses/by-sa/2.0/ 5271 6 0bc8a139b0 b291292554 jpg 0
Total number of elements in originalFlickrMeta: 100

```

Explanation: We check the first lines and total count. This confirms the RDD loaded correctly.

Question 2

Task: Transform the RDD[String] in RDD[Picture] using the Picture class. Only keep interesting pictures having a valid country and tags. To check your program, display 10 elements.

Code:

```

val splitRecords = originalFlickrMeta.map(line => line.split("\t"))
val pictures: RDD[Picture] = splitRecords.flatMap(fields => Try(Picture(fields)).
  toOption)
val interestingPictures: RDD[Picture] = pictures.filter(pic => pic.c != null && pic.
  hasTags)

printScope("Question 2 - First 10 elements from interesting RDD[Picture]")
interestingPictures.take(10).foreach(println)

```

Output:

```

Question 2 - First 10 elements from interesting RDD[Picture]
Picture([Ljava.lang.String;@57877e5)
Picture([Ljava.lang.String;@323aeae)
Picture([Ljava.lang.String;@7ecee6ae)
Picture([Ljava.lang.String;@4ed9acf0)
Picture([Ljava.lang.String;@8583d91)
Picture([Ljava.lang.String;@1610b4ea)
Picture([Ljava.lang.String;@1fe2db10)
Picture([Ljava.lang.String;@1cdf4a30)
Picture([Ljava.lang.String;@55a2588f)
Picture([Ljava.lang.String;@46860135)

```

Explanation: We convert text lines to Picture objects and keep only interesting ones. The printed objects show that conversion works.

Question 3

Task: Group the images by country (groupBy). Print the list of images corresponding to the first country. What is the type of this RDD?

Code:

```
val picturesByCountry = interestingPictures.groupBy(_.c)

printScope("Question 3 - First country and its corresponding list of images")
val firstCountryGroup = picturesByCountry.take(1)
firstCountryGroup.foreach(println)
```

Output:

```
Question 3 - First country and its corresponding list of images
(ML,CompactBuffer(Picture([Ljava.lang.String;@1031f334), Picture([Ljava.lang.String;
  @4e0d4d9a), Picture([Ljava.lang.String;@22a315fd), Picture([Ljava.lang.String;
  @594e1439), Picture([Ljava.lang.String;@4679e477), Picture([Ljava.lang.String;
  @62bc00bf), Picture([Ljava.lang.String;@588afde8), Picture([Ljava.lang.String;
  @714cc51c), Picture([Ljava.lang.String;@781fcd1e), Picture([Ljava.lang.String;
  @7a24835a), Picture([Ljava.lang.String;@754e444b), Picture([Ljava.lang.String;
  @623d1bdb), Picture([Ljava.lang.String;@64cc4dbf), Picture([Ljava.lang.String;
  @3a2c2c2f), Picture([Ljava.lang.String;@3dc74b3d), Picture([Ljava.lang.String;
  @372e6c9d), Picture([Ljava.lang.String;@31897e72), Picture([Ljava.lang.String;
  @6a1f6138), Picture([Ljava.lang.String;@6d8641e5), Picture([Ljava.lang.String;
  @2988924c), Picture([Ljava.lang.String;@538813e1), Picture([Ljava.lang.String;
  @428f8e09), Picture([Ljava.lang.String;@1dd946d4), Picture([Ljava.lang.String;
  @702eadd1), Picture([Ljava.lang.String;@63aabb2e), Picture([Ljava.lang.String;
  @5c6a3aa4), Picture([Ljava.lang.String;@b52281b), Picture([Ljava.lang.String;
  @e974c85)))
```

Type of this RDD is: RDD[(Country, Iterable[Picture])]

Explanation: We group pictures by country. Each key is one country and each value is a list of pictures from that country.

Question 4

Task: Build an RDD containing pairs where the first element is a country and the second element is the list of tags used on pictures taken in this country.

Code:

```
val tagsByCountry = picturesByCountry.map(item => {
  val country = item._1
  val pics = item._2
  val allTags = pics.flatMap(pic => pic.userTags).toList
  (country, allTags)
})

printScope("Question 4 - First 10 (Country, List[Tags]) pairs")
tagsByCountry.take(10).foreach(println)
```

Output:

```
Question 4 - First 10 (Country, List[Tags]) pairs
(ML,List(yosemite, yosemite, yosemite, yosemite, yosemite, yosemite,
  yosemite, canada square park, canary wharf, jiving lindy hoppers, pasadena roof
  orchestra, twilight delights, boat, dune, gao, mali, niger, river, sahara, sand,
  mali, yosemite, canada square park, canary wharf, jiving lindy hoppers, pasadena
  roof orchestra, twilight delights, canada square park, canary wharf, jiving lindy
  hoppers, pasadena roof orchestra, twilight delights, mali, canada square park,
  canary wharf, jiving lindy hoppers, pasadena roof orchestra, twilight delights,
  mali, gao, mali, man, nomad, sahara, tuareg, africa, desierto, islam, mali,
  mezquitas, niger, rio niger, tombuctú, viajes, africa, desierto, islam, mali,
```

```

mezquitas, niger, rio niger, tombuctú, viajes, africa, desierto, islam, mali,
mercado tombuctú, mezquitas, niger, rio niger, viajes, africa, desierto, islam,
mali, mezquitas, niger, rio niger, tombuctú, viajes, africa, desierto, islam, mali
, mezquitas, niger, rio niger, tombuctú, viajes, africa, desierto, islam, mali,
mezquitas, niger, rio niger, tuaregs tombuctú, viajes, africa, desierto, islam,
mali, mezquitas, niger, rio niger, tuaregs tombuctú, viajes, 4x4, africa, desierto
, islam, mali, mezquitas, niger, pescados, rio niger, transbordador tombuctú,
viajes, áfrica, desierto, islam, mali, mezquita, niger, rio niger, tombuctú,
tuaregs, viajes, africa, desierto, islam, mali, mezquitas, niger, rio niger,
tombuctú, viajes))
(UV,List(aids, art education, ghana, hiv, hiv/aids, hiv prevention, lotos collective,
malina de carlo, roberto sanchez-camus, youth visions, aids, art education, ghana,
hiv, hiv/aids, hiv prevention, lotos collective, malina de carlo, roberto sanchez
-camus, youth visions, africa, ghana, idds, night, dhf, ghana, gspd, aids, art
education, ghana, hiv, hiv/aids, hiv prevention, lotos collective, malina de carlo
, roberto sanchez-camus, youth visions, africa, bedroom, bolga, ghana, hazwan,
africa, ghana, aids, art education, ghana, hiv, hiv/aids, hiv prevention, lotos
collective, malina de carlo, roberto sanchez-camus, youth visions, burkinafaso,
electricity, informal, mfp, moulin, burkinafaso, electricity, informal, mfp,
moulin, burkinafaso, electricity, informal, mfp, moulin, burkinafaso, electricity,
informal, mfp, moulin, burkinafaso, electricity, informal, mfp, moulin, afrika,
burkina faso, entwicklungshilfe, patenschaft, afrika, burkina faso,
entwicklungshilfe, patenschaft, adaptation, agriculture, burkina faso, ccafs,
cgia, cgiaclimate, community, demi-lune, desert, drought, drylands, education,
farmer, farmers organisations, gourcy, innovation, participation, sahel, west
africa, zai, zodoma, 12scatti, burkina faso, 2007, africa, afrika, afrique,
afrique de l'ouest, burkina, burkina faso, dori, faso, img_8641.jpg, travel, west
africa, westafrika, adobe, africa, afrique, banco, bani, burkina, burkina_faso,
faso, fulani, guillaume_colin, mudbrick, peul, sahel, afrique, blanc, burkinafaso,
bw, et, noir, ouagadougou, 2007, africa, afrika, afrique, afrique de l'ouest,
burkina, burkina faso, dori, faso, img_8643.jpg, travel, west africa, westafrika,
yosemite, adobe, africa, afrique, banco, bani, burkina, burkina_faso, faso,
guillaume_colin, mosquée, yosemite, afrique, blanc, burkinafaso, bw, et, noir,
ouagadougou, 2007, africa, afrika, afrique, afrique de l'ouest, beggar, burkina,
burkina faso, dori, faso, fulani, travel, west africa, westafrika, 2007, africa,
afrika, afrique, afrique de l'ouest, burkina, burkina faso, dori, faso, img_8649.
jpg, travel, west africa, westafrika, 2007, africa, afrika, afrique, afrique de l'
ouest, burkina, burkina faso, dori, faso, img_8602.jpg, travel, west africa,
westafrika, burkina-faso, gorom-gorom, burkina-faso, gorom-gorom, burkina-faso,
oursi, burkina-faso, oursi, burkina-faso, oursi, burkina-faso, oursi, burkina-faso
, gorom-gorom, burkina-faso, gorom-gorom, burkina-faso, oursi))
(AG,List(, , , , alger, algeria, amazigh culture, hoggar, la culture amazighe,
tamanrasset, touareg, , , , , alger, algeria, amazigh culture, hoggar, la
culture amazighe, tamanrasset, , , , , alger, algeria, amazigh culture, hoggar))
(BN,List(africa, ghana, idds, navrongo, africa, ghana, idds, rice, single mothers,
ghana, lab, ghana, lab, ghana, lab, ghana, lab, ghana, lab))

```

Explanation: We create pairs of country and all tags used in that country. This helps us prepare for tag counting.

Question 5

Task: Build an RDD of type `RDD[(Country, Map[String, Int])]` by associating each tag with its frequency.

Code:

```
val tagFrequencyByCountry = tagsByCountry.map(item => {
```

```

val country = item._1
val tags = item._2

val tagFrequencies = tags.groupBy(x => x).map(tagGroup => {
  val tag = tagGroup._1
  val occurrences = tagGroup._2
  (tag, occurrences.size)
})

(country, tagFrequencies)
})

printScope("Question 5 - First 10 (Country, Map[Tag -> Frequency]) pairs")
tagFrequencyByCountry.take(10).foreach(println)

```

Output:

```

Question 5 - First 10 (Country, Map[Tag -> Frequency]) pairs
(ML,Map(sand -> 1, canary wharf -> 4, dune -> 1, mezquitas -> 9, tuaregs -> 1, gao ->
  2, nomad -> 1, transbordador tombuctú -> 1, river -> 1, yosemite -> 9, rio niger
-> 10, man -> 1, boat -> 1, mali -> 15, pasadena roof orchestra -> 4, mezquita ->
  1, africa -> 9, twilight delights -> 4, viajes -> 10, jiving lindy hoppers -> 4, 4
x4 -> 1, tombuctú -> 6, áfrica -> 1, desierto -> 10, mercado tombuctú -> 1, niger
-> 11, tuaregs tombuctú -> 2, islam -> 10, canada square park -> 4, pescados -> 1,
  tuareg -> 1, sahara -> 2))
(UV,Map(burkina_faso -> 2, patenschaft -> 2, img_8602.jpg -> 1, community -> 1, zai ->
  1, drylands -> 1, westafrika -> 5, burkina-faso -> 9, aids -> 4, bani -> 2,
  img_8643.jpg -> 1, hiv prevention -> 4, moulin -> 5, dori -> 5, mfp -> 5, desert
-> 1, entwicklungshilfe -> 2, noir -> 2, 2007 -> 5, oursi -> 5, 12scatti -> 1, bw
-> 2, gspd -> 1, beggar -> 1, farmers organisations -> 1, peul -> 1, bedroom -> 1,
  mosquée -> 1, burkina faso -> 9, hazwan -> 1, adobe -> 2, night -> 1, malina de
carlo -> 4, zodoma -> 1, yosemite -> 2, burkinafaso -> 7, afrique -> 9, ghana ->
  8, burkina -> 7, et -> 2, west africa -> 6, mudbrick -> 1, demi-lune -> 1,
  img_8649.jpg -> 1, participation -> 1, agriculture -> 1, travel -> 5, banco -> 2,
  africa -> 10, ouagadougou -> 2, blanc -> 2, education -> 1, lotos collective -> 4,
  cgiarclimate -> 1, faso -> 7, roberto sanchez-camus -> 4, idds -> 1, innovation
-> 1, adaptation -> 1, drought -> 1, sahel -> 2, ccafs -> 1, bolga -> 1, cgiar ->
  1, informal -> 5, farmer -> 1, gourcy -> 1, hiv/aids -> 4, dhf -> 1, youth visions
-> 4, art education -> 4, img_8641.jpg -> 1, gorom-gorom -> 4, afrika -> 7,
  electricity -> 5, guillaume_colin -> 2, afrique de l'ouest -> 5, hiv -> 4, fulani
-> 2))
(AG,Map(-> 3, -> 3, tamanrasset -> 2, -> 3, -> 3, algeria -> 3, touareg -> 1, alger
-> 3, hoggar -> 3, la culture amazighe -> 2, -> 3, amazigh culture -> 3))
(BN,Map(lab -> 5, ghana -> 7, rice -> 1, single mothers -> 1, africa -> 2, idds -> 2,
  navrongo -> 1))

```

Explanation: We count how many times each tag appears in each country. The map gives tag frequency per country.

Note: In the outputs above (particularly for country AG in Question 4 and Question 5), there are Arabic characters present in the dataset. Because of LaTeX font rendering and encoding constraints for non-Latin scripts, these characters might appear empty or improperly formatted in the generated PDF.

Overall Summary

Part 1 uses Spark SQL and DataFrame operations. We select fields, filter data, check plans, join with license data, and save final CSV.

Part 2 uses Spark RDD operations. We parse records into Picture objects, group by country, collect tags, and count tag frequency.

The results show the pipeline works from raw data to final structured output.

Submitted Files

As required, the code is kept in assignment files only:

- `Part1.scala`
- `Part2.scala`
- this report file